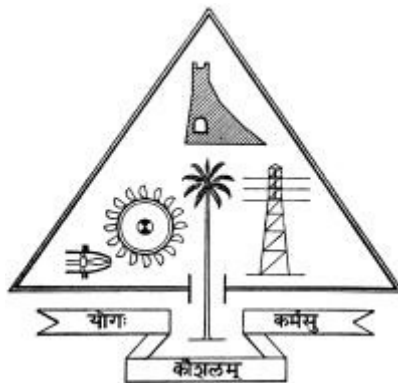


SMARTEDIT: A HUMAN-IN-THE-LOOP AI FRAMEWORK FOR MULTI-FOOTAGE VIDEO EDITING

*Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of Master of Computer Applications of the APJ Abdul
Kalam Technological University*

Submitted by

**SUDHI BABU
(TCR25MCA-2052)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

MARCH 2026

ABSTRACT

Video editing is a time-consuming process that requires editors to manually review footage, remove unwanted segments, arrange clips, and construct a coherent timeline. While recent AI-powered editing tools automate parts of this workflow, they often offer limited user control and can produce edits that do not match the creator's intended style. This creates a need for an intelligent editing system that combines the speed of artificial intelligence with the creativity and decision-making of human editors.

This project proposes SmartEdit, an AI-assisted Human-in-the-Loop (HITL) video editing system designed to accelerate traditional video editing while keeping the user in control of the final output. SmartEdit allows users to describe their desired editing style through natural language prompts, specifying the type of video and the required duration. Based on these instructions, the system analyzes video and audio content to perform tasks such as shaky footage detection, silence-based trimming, unnecessary clip removal, rough cut generation, and intelligent clip arrangement. Video analysis is performed using OpenCV, audio analysis using Librosa, and a Small Language Model (SLM) interprets user prompts to generate editing decisions. The application is built as desktop software using Qt (C++) with Python-based AI modules, and FFmpeg handles video processing and export.

Unlike fully automated editing tools, SmartEdit follows a Human-in-the-Loop approach by presenting the AI-generated rough edit for user review. Editors can preview the generated timeline, accept or reject AI suggestions, manually modify clips, rearrange the timeline, and apply further refinements before exporting the final video. This collaborative workflow improves editing efficiency while preserving creative control. The proposed system aims to

significantly reduce editing time, improve productivity, and simplify rough-cut creation for content creators, YouTubers, wedding videographers, news editors, and multimedia professionals. By combining AI-assisted automation with human decision-making, SmartEdit offers a practical and efficient solution for modern video editing workflows.

CONTENTS

1	INTRODUCTION	1
2	LITERATURE REVIEW	4
3	ENVIRONMENTAL STUDY	8
3.1	System Configuration	8
3.1.1	Hardware Requirements	8
3.1.2	Software Requirements	9
3.1.3	Software Requirements for Development	10
3.2	Software Specification	12
3.2.1	Qt 6 Framework	12
3.2.2	Python 3.11	12
3.2.3	OpenCV	13
3.2.4	Librosa	13
3.2.5	Small Language Model (SLM)	13
3.2.6	FFmpeg	13
3.2.7	Git	14
3.2.8	Human-in-the-Loop Editing Module	14
4	SYSTEM ANALYSIS	15
4.1	Requirements Analysis	16
4.2	Existing System	17
4.3	Limitations of Existing System	19
4.4	Proposed System	20
4.5	Advantages of Proposed System	22
4.6	Functional Requirements	23
4.6.1	Video Import and Project Creation	23
4.6.2	Prompt-Based Editing	24
4.6.3	Video Analysis	24
4.6.4	Audio Analysis	25
4.6.5	AI-Assisted Rough Cut Generation	25
4.6.6	Human-in-the-Loop Review	25
4.6.7	Manual Timeline Editing	26
4.6.8	Video Rendering and Export	26
4.7	Non-Functional Requirements	26
4.7.1	Performance	26
4.7.2	Usability	27
4.7.3	Accuracy and Reliability	27
4.7.4	Security and Data Privacy	27
4.7.5	Maintainability and Extensibility	27
4.7.6	Scalability	28
4.7.7	Human-in-the-Loop Editing	28
4.8	Feasibility Study	28

4.8.1	Technical Feasibility	28
4.8.2	Operational Feasibility	29
4.8.3	Economical Feasibility	29
	Bibliography	30
	Bibliography	30

CHAPTER 1

INTRODUCTION

Video content has become one of the fastest-growing forms of digital communication. Every day, millions of videos are created for platforms such as YouTube, Instagram, Facebook, educational websites, news media, and business promotions. As the demand for video content continues to increase, creators are expected to produce high-quality videos within a short time. However, video editing remains one of the most time-consuming stages of content production. Editors must manually review large amounts of raw footage, remove unusable clips, eliminate silent portions, trim unnecessary sections, arrange clips in the correct sequence, and prepare a rough cut before beginning the creative editing process.

Although modern video editing software provides several AI-powered features such as scene detection, automatic captions, silence removal, and object tracking, these features usually work independently and require considerable manual interaction. Fully automated AI editing tools can generate edited videos quickly, but they often fail to understand the editor's creative intention, editing style, or storytelling requirements. As a result, editors still spend significant time correcting AI-generated edits, reducing the overall benefit of automation.

One of the major challenges in video editing is creating the initial rough cut. This stage involves identifying useful footage, removing shaky or poor-quality clips, trimming silent sections, and arranging clips into a meaningful sequence. These tasks are repetitive and consume a large portion of the total editing time, even though they follow predictable patterns that can be analyzed using Artificial Intelligence. Therefore, there is a growing need

for an intelligent editing assistant that can automate repetitive editing tasks while allowing editors to retain complete creative control over the final video.

Human-in-the-Loop (HITL) Artificial Intelligence provides an effective solution to this challenge. Instead of replacing the editor, HITL combines the speed of AI with human decision-making. The AI system performs repetitive analytical tasks and generates editing suggestions, while the user reviews, accepts, rejects, or modifies those suggestions before producing the final output. This collaborative workflow improves editing efficiency without sacrificing creativity or editorial control.

This project proposes **SmartEdit**, an AI-assisted Human-in-the-Loop video editing system designed to accelerate the rough-cut editing process. The system allows users to describe the required editing style and expected video duration using natural language prompts. A Small Language Model (SLM) interprets these prompts and converts them into editing instructions. Based on these instructions, SmartEdit automatically analyzes video and audio content to generate an initial rough edit.

SmartEdit uses OpenCV to perform computer vision tasks such as detecting shaky footage and identifying low-quality clips. Audio processing is performed using the Librosa library to detect silence and remove unnecessary pauses from the video. The processed clips are then automatically arranged into a rough timeline according to the user's editing requirements. The desktop application is developed using the Qt Framework with C++ for the graphical user interface, Python for AI processing modules, and FFmpeg for video processing and final export.

Unlike fully automated editing software, SmartEdit provides an interactive editing workflow. After generating the rough cut, the system presents the editing timeline to the user for review. Editors can preview the generated sequence, accept or reject AI suggestions, rearrange clips, perform additional manual editing, and export the final video. This review-and-refinement approach ensures that AI serves as an intelligent assistant while the user remains responsible for all creative decisions.

The scope of SmartEdit includes natural language-based editing instructions, automatic shaky footage detection, silence detection and trimming, AI-assisted rough-cut generation, automatic clip arrangement, interactive timeline editing, user approval of AI suggestions, manual refinement, and final video export. Advanced post-production features such as colour grading, visual effects, motion graphics, advanced audio mixing, and multi-camera synchronization are outside the scope of this project, as the primary objective is to reduce the time required to create an accurate rough cut.

By integrating Artificial Intelligence, Computer Vision, audio signal processing, and Human-in-the-Loop interaction into a single desktop application, SmartEdit aims to simplify the video editing workflow, reduce manual effort, and improve editing productivity. The proposed system provides an efficient and practical solution for YouTubers, wedding videographers, content creators, news editors, and multimedia professionals who require faster rough-cut generation while maintaining complete creative control over the final video. ““

CHAPTER 2

LITERATURE REVIEW

The rapid advancement of Artificial Intelligence (AI), Computer Vision, Natural Language Processing (NLP), and multimedia processing technologies has significantly transformed modern video production and editing workflows. The increasing popularity of digital content creation across platforms such as YouTube, Instagram, Facebook, and professional media production has created a growing demand for intelligent editing solutions capable of reducing manual editing effort while maintaining high-quality output. Recent research has therefore focused on integrating AI with video editing systems to automate repetitive tasks such as scene detection, clip selection, silence removal, highlight generation, and content understanding. Despite these developments, most existing systems remain either fully manual or completely automated, providing limited opportunities for editors to interact with or refine AI-generated editing decisions.

Traditional video editing software primarily relies on manual interaction through timeline-based interfaces, where editors are responsible for reviewing raw footage, removing unwanted segments, trimming silent portions, arranging clips, and producing the final video. Although professional editing applications such as Adobe Premiere Pro and DaVinci Resolve provide AI-assisted features including scene detection, speech enhancement, and automatic reframing, these capabilities operate as independent tools that still require considerable manual intervention. Consequently, editors continue to spend a significant amount of time creating rough cuts before beginning creative post-production.

Computer Vision has become one of the fundamental technologies

enabling intelligent video editing. Zeng Boyi and Hou Jianzhong [1] investigated the application of computer vision in film and television production, demonstrating that AI-based visual analysis can automatically identify scene characteristics, camera movements, and video quality, thereby reducing manual editing effort. Similarly, Zhang Kang et al. [2] proposed an AI-based automatic short video production framework capable of analyzing visual content and generating edited videos automatically. These studies demonstrate the effectiveness of computer vision in reducing editing effort; however, they primarily focus on automatic content generation and provide limited opportunities for user interaction during the editing process.

Recent research has also explored multimodal learning techniques for understanding video content more effectively. Li Zijian [3] investigated multimodal emotion recognition by combining visual and audio features through deep learning, demonstrating improved recognition accuracy compared with single-modality approaches. Likewise, Zeng Boyi and Zheng Bing [4] presented a computer vision-based multimodal emotion recognition framework capable of analyzing both visual and auditory information simultaneously. Although these approaches significantly improve content understanding, they mainly concentrate on emotion recognition rather than assisting editors in generating practical rough cuts for video production.

Automatic highlight generation has also received considerable research attention. The work titled *Video Highlight Segment Recognition and Automatic Editing Method Based on Spatio-Temporal Convolution Network and Multimodal Emotion Alignment* [5] proposed a deep learning framework capable of recognizing highlight segments using spatio-temporal convolution networks and multimodal emotion alignment. The proposed approach effectively identifies important events within video sequences and generates highlight videos automatically. Nevertheless, the editing workflow remains fully automated, offering minimal transparency and limited opportunities for users to modify AI-generated editing decisions.

Natural Language Processing has recently been introduced into

multimedia applications to simplify user interaction with AI systems. Instead of manually configuring editing parameters, researchers have investigated the use of language models for understanding user intentions expressed through natural language. This approach enables users to describe editing requirements using ordinary language rather than complicated editing settings, thereby improving accessibility for non-professional editors. However, most existing editing systems still lack comprehensive prompt-based editing capabilities integrated directly with video analysis and timeline generation.

Audio signal processing has similarly contributed to intelligent editing by enabling automatic silence detection, speech segmentation, and audio enhancement. Libraries such as Librosa provide reliable techniques for extracting audio features including Root Mean Square (RMS) energy, spectral properties, and silence intervals, making automatic silence removal increasingly practical. Although silence detection is commonly available as an independent feature, it is rarely integrated with visual analysis and user-driven editing preferences to generate complete AI-assisted rough cuts.

Human–Computer Interaction research has further demonstrated that intelligent systems should collaborate with users rather than replacing them entirely. Gao Meng, Han Jinshan, and Zeng Zhigang [6] emphasized that future intelligent editing systems should combine AI automation with human creativity instead of relying exclusively on fully automated editing. Similarly, Meng Xiang and Li Sha [7] highlighted the importance of maintaining narrative consistency and editing rhythm through intelligent editing assistance. These studies indicate that collaborative editing environments represent a promising direction for future multimedia production.

Although significant progress has been achieved in computer vision, multimodal learning, audio analysis, and AI-assisted editing, these technologies have largely evolved independently. Existing research primarily focuses on automatic scene detection, emotion recognition, highlight generation, or intelligent content understanding. Most proposed

systems generate editing results automatically without providing sufficient opportunities for users to review, modify, or personalize AI-generated editing decisions. Furthermore, very limited research combines natural language prompt interpretation, computer vision, audio signal processing, AI-assisted rough-cut generation, and Human-in-the-Loop (HITL) editing within a unified desktop application.

This research gap motivates the development of the proposed **SmartEdit** system, an AI-assisted Human-in-the-Loop video editing framework that integrates natural language prompt interpretation using a Small Language Model (SLM), computer vision-based video analysis through OpenCV, audio analysis using Librosa, intelligent rough-cut generation, and an interactive timeline for user review and refinement. Unlike existing AI-powered editing tools that operate as fully automated black-box systems, SmartEdit allows editors to preview, accept, reject, rearrange, and manually refine AI-generated edits before exporting the final video. By combining AI automation with human creativity, SmartEdit aims to significantly reduce the time required to create rough cuts while preserving complete editorial control over the final output.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter presents the hardware and software environment required for the development, execution, and deployment of the proposed **SmartEdit** system. The application integrates desktop application development, computer vision, audio signal processing, natural language understanding, and multimedia processing to provide an AI-assisted Human-in-the-Loop video editing workflow. The environment is designed to support real-time video playback, AI-based video analysis, prompt interpretation, and efficient video rendering while maintaining a responsive user experience.

3.1 System Configuration

The SmartEdit system consists of a hybrid software architecture in which the desktop application is developed using Qt (C++), while AI-based processing modules are implemented in Python. The system utilizes OpenCV for video analysis, Librosa for audio processing, a Small Language Model (SLM) for interpreting user prompts, and FFmpeg for video encoding, rendering, and export.

3.1.1 Hardware Requirements

The following hardware configuration is recommended to ensure smooth execution of video processing, AI inference, and multimedia rendering tasks.

- **Processor:** Intel® Core™ i5 (11th Generation or higher) or AMD Ryzen™ 5 Processor or above for efficient video processing and AI computations.
- **Memory:** Minimum 8 GB RAM (16 GB recommended) to support

simultaneous execution of the desktop application, AI modules, and multimedia processing.

- **Storage:** Minimum 100 GB SSD free space for storing video files, temporary processing data, AI models, and exported videos.
- **Graphics:** NVIDIA GeForce GTX 1650 or higher (or equivalent GPU) is recommended for improved video rendering and future GPU-accelerated AI processing.
- **Display:** Full HD (1920 × 1080) monitor for comfortable video preview, timeline editing, and user interaction.

3.1.2 *Software Requirements*

The software requirements specify the operating environment, programming languages, development frameworks, multimedia libraries, and AI tools required for the successful development and execution of the SmartEdit system. These software components work together to support desktop application development, video processing, computer vision, audio analysis, natural language understanding, and multimedia rendering. Proper compatibility among these components ensures efficient communication between the user interface and the AI processing modules.

The key software requirements are detailed as follows:

- **Operating System:** Microsoft Windows 10/11 (64-bit) is recommended for development and deployment. The application can also be adapted for Linux-based operating systems with Qt support.
- **Programming Languages:** C++ (C++17 or later) for desktop application development and Python 3.11 or later for implementing AI processing modules.
- **Development Framework:** Qt 6 Framework for developing the graphical user interface, video playback, timeline editor, and overall desktop application.

- **Computer Vision Library:** OpenCV 4.x for video frame analysis, shaky footage detection, frame comparison, and clip processing.
- **Audio Processing Library:** Librosa for audio feature extraction, silence detection, and speech activity analysis.
- **Artificial Intelligence Model:** Small Language Model (SLM) for interpreting natural language editing prompts and generating intelligent editing instructions.
- **Video Processing Engine:** FFmpeg for video decoding, trimming, encoding, rendering, and exporting the final edited video.
- **Development Tools:** Qt Creator, Visual Studio Code, Python virtual environment, and Git for application development, debugging, dependency management, and version control.

3.1.3 Software Requirements for Development

In addition to the software required for system execution, various development tools are essential for designing, implementing, testing, debugging, and maintaining the SmartEdit application. These tools facilitate desktop application development, AI module integration, multimedia processing, and source code management. They also support collaboration, version tracking, and efficient testing throughout the software development lifecycle.

The key development software requirements are detailed as follows:

- **Operating System:** Microsoft Windows 10/11 (64-bit) is recommended for development. Ubuntu 22.04 LTS or later may also be used for developing and testing Python-based AI modules.
- **Programming Languages:** C++17 or later for developing the desktop application and Python 3.11 or later for implementing AI-based video and audio analysis modules.

- **Development Environment (IDE):** Qt Creator for developing the Qt desktop application, and Visual Studio Code for Python development, debugging, and AI module integration.
- **Frameworks and Libraries:** Qt 6 Framework, OpenCV, Librosa, FFmpeg, and a Small Language Model (SLM) runtime for implementing multimedia processing and AI-assisted editing features.
- **Version Control:** Git distributed version control system for source code management, collaboration, and project maintenance.

Development Tools and Frameworks:

- **Qt 6 Framework:** Used for developing the desktop application's graphical user interface (GUI), timeline editor, media playback, and overall user interaction.
- **OpenCV Library:** Utilized for computer vision tasks such as video frame analysis, shaky footage detection, frame comparison, and clip quality assessment.
- **Librosa Library:** Used for audio signal processing, including silence detection, audio feature extraction, and speech activity analysis for automatic trimming.
- **Small Language Model (SLM):** Responsible for interpreting natural language editing prompts and generating intelligent editing instructions based on user requirements.
- **FFmpeg Multimedia Framework:** Employed for video decoding, encoding, trimming, rendering, and exporting the final edited video with high-quality multimedia support.
- **Python Libraries:** Libraries such as NumPy and MoviePy are used to support AI processing, video manipulation, and communication between the Python modules and the Qt application.

- **Git Version Control:** Used for source code management, version tracking, and collaborative software development.

3.2 Software Specification

The following section describes the major software components and technologies employed in the development of the proposed SmartEdit system. Each component performs a specific function within the AI-assisted video editing pipeline, ranging from desktop application development and multimedia processing to computer vision, audio analysis, and natural language understanding. The integration of these technologies provides a scalable, modular, and efficient platform for accelerating the traditional video editing workflow while maintaining human control over editing decisions.

3.2.1 Qt 6 Framework

The Qt 6 Framework serves as the primary platform for developing the SmartEdit desktop application. It provides the graphical user interface, video playback controls, timeline editor, project management features, and user interaction components. Qt's cross-platform architecture and multimedia support enable responsive playback and smooth interaction with large video files. It also acts as the communication layer between the desktop application and the AI modules developed in Python.

3.2.2 Python 3.11

Python functions as the AI processing engine of SmartEdit. It coordinates video analysis, audio processing, prompt interpretation, and communication with the desktop application. Python's extensive ecosystem of scientific computing and machine learning libraries enables efficient implementation of multimedia analysis algorithms while providing flexibility for integrating future AI models and advanced editing capabilities.

3.2.3 OpenCV

OpenCV forms the computer vision backbone of the SmartEdit system. It is responsible for analyzing video frames, detecting shaky footage, identifying blurred or low-quality clips, comparing frame similarity, and extracting visual information required for generating an intelligent rough cut. The extracted visual features enable the system to recommend suitable clips while reducing the amount of manual footage review required by the editor.

3.2.4 Librosa

Librosa is employed for audio signal processing within SmartEdit. The library extracts audio features such as Root Mean Square (RMS) energy and performs silence detection to identify unwanted pauses in recorded videos. By automatically locating silent regions, the system assists in silence-based trimming, reducing repetitive editing work and improving the pacing of the generated rough cut.

3.2.5 Small Language Model (SLM)

The Small Language Model (SLM) provides natural language understanding capabilities for the SmartEdit system. It interprets user prompts describing the desired editing style, target duration, and clip arrangement preferences, and converts them into structured editing instructions. This enables users to control the editing workflow using simple natural language instead of manually configuring complex editing parameters, thereby improving accessibility and user experience.

3.2.6 FFmpeg

FFmpeg serves as the multimedia processing engine responsible for video decoding, encoding, trimming, rendering, and final export. It performs efficient processing of various video formats while ensuring high-quality output. FFmpeg also enables seamless communication between the AI-generated editing decisions and the final rendered video, making it a critical

component of the overall editing pipeline.

3.2.7 *Git*

Git is used as the version control system throughout the development of SmartEdit. It supports source code management, tracks project revisions, facilitates collaboration, and maintains development history. Version control enables systematic software maintenance and simplifies testing, debugging, and future feature enhancements.

3.2.8 *Human-in-the-Loop Editing Module*

The Human-in-the-Loop (HITL) module is the central component that distinguishes SmartEdit from conventional AI-powered editing tools. Instead of automatically producing a final video, the module presents an AI-generated rough edit through the timeline interface, allowing users to preview, accept, reject, rearrange, or modify editing suggestions before export. This collaborative workflow combines the efficiency of AI automation with the creativity and judgment of the human editor, ensuring that the final video reflects the user's intended editing style.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is a crucial phase in the software development life cycle that focuses on understanding the existing problem, identifying user requirements, and designing an effective solution before implementation. In the context of SmartEdit, the primary objective is to reduce the time and effort involved in traditional video editing by integrating Artificial Intelligence (AI) with a Human-in-the-Loop (HITL) editing workflow. The analysis phase examines the limitations of existing video editing software, studies the requirements of content creators and professional editors, and defines a system capable of automating repetitive editing tasks while preserving human creativity and control.

Traditional video editing requires editors to manually review raw footage, remove unwanted clips, trim silent sections, detect unstable shots, and arrange clips into a meaningful sequence before producing the final output. Although modern AI-powered editing applications provide automation for some of these tasks, they often function as fully automated systems that offer limited transparency and minimal user control over editing decisions. Consequently, editors frequently spend additional time modifying AI-generated outputs to match their intended storytelling style and creative vision. The system analysis therefore focuses on bridging the gap between manual editing and complete automation by proposing an intelligent assistant that generates a rough cut while allowing users to review, accept, reject, and refine AI-generated suggestions.

The SmartEdit system integrates computer vision, audio signal processing, natural language understanding, and desktop multimedia technologies to

create an efficient editing workflow. The system interprets user editing requirements through natural language prompts, analyzes video and audio content to detect unnecessary footage and silent segments, generates an AI-assisted rough edit, and presents the results through an interactive timeline interface for human review. By combining AI-driven automation with human decision-making, the proposed system aims to improve editing productivity, reduce repetitive manual work, and provide a practical, flexible, and user-centric solution for modern video editing.

4.1 Requirements Analysis

Requirement analysis is a crucial phase in the software development life cycle that focuses on identifying, documenting, and analyzing the functional and technical needs of a system before design and implementation. For the proposed RustAuditAI project, which aims to perform intra-procedural software quality intelligence, vulnerability detection, and explainable patch generation within individual Rust functions, this phase plays a vital role in defining the system's scope and expected behavior.

The primary requirement of the system is to support source code blocks written in the Rust programming language. Individual functions must be parsed into a structured abstract form using compiler-level front-ends, specifically the syn crate ecosystem. This compiler-level syntax extraction enables language-idiom validation and forms the foundation for consistent, intra-procedural data-flow and lifetime tracking without requiring full workspace compilation binaries.

Another key requirement is the integration of Large Language Models for semantic code reasoning and software quality analysis. The system must be capable of extracting localized graph properties from the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Function-Level Ownership Graph (FLOG), and providing them as structured context inputs to LLMs for identifying safety defects, allocation bottlenecks, and logical vulnerabilities. In addition to detection, the system is required to generate natural-language

explanations using Explainable AI (XAI) techniques, ensuring that developers understand the root cause, execution flow, and structural impact of each detected quality degradation pattern.

The system must also support automatic patch generation, where secure and optimized code fixes are produced for identified function flaws. These fixes should be presented in a clear old-versus-new diff format to allow developers to review, apply, or refine the suggested patches. Furthermore, visualization support is required to generate interactive, self-contained functional Control Flow Graphs that visually illustrate the propagation of variables, resource allocations, and the specific structural branches leading to code degradation.

In addition to functional requirements, several non-functional requirements must be satisfied. The system should provide highly efficient intra-procedural analysis with rapid execution times suitable for active development loops. It must maintain strict determinism in its mathematical scoring engine and be extensible enough to incorporate additional performance or safety criteria in the future. Usability is also a critical requirement, with metrics and reports designed to be clear, educational, and developer-friendly.

This requirement analysis ensures that the RustAuditAI system is aligned with its intended objectives and provides a reliable foundation for subsequent system design, implementation, and evaluation.

4.2 Existing System

Existing video editing systems are primarily designed either as professional manual editing applications or fully AI-powered automatic editors. Professional software provides extensive creative control and advanced editing features but requires editors to manually perform repetitive tasks such as reviewing footage, removing unwanted clips, trimming silent sections, detecting unstable shots, and arranging clips into a meaningful sequence. On the other hand, AI-assisted editing tools automate several

editing operations, including scene detection, silence removal, highlight generation, and template-based editing. Although these systems reduce editing time, they often operate as black-box solutions with limited transparency and minimal user involvement during the decision-making process. Consequently, editors frequently need to modify AI-generated outputs to match their desired storytelling style, pacing, and creative vision.

Another major limitation of existing systems is the absence of an effective Human-in-the-Loop (HITL) workflow. Most AI-based editors automatically generate the final video without allowing users to review or refine individual editing decisions before export. Furthermore, existing tools rarely provide prompt-based editing capabilities that enable users to describe their editing requirements using natural language. This lack of flexibility reduces user control and limits the adaptability of AI-generated edits to different editing styles. Some of the widely used existing systems include:

- **Adobe Premiere Pro:** Adobe Premiere Pro is one of the most widely used professional video editing applications. It provides comprehensive editing features including timeline-based editing, transitions, color correction, audio mixing, and AI-powered tools such as Auto Reframe and Speech Enhancement. Although these features improve productivity, editors are still required to manually organize footage, perform rough-cut creation, and refine AI-generated suggestions. The software is feature-rich but requires considerable editing expertise and significant manual effort.
- **DaVinci Resolve:** DaVinci Resolve combines professional video editing, visual effects, color grading, and audio post-production within a single application. Its Neural Engine incorporates AI capabilities such as object detection, facial recognition, scene cut detection, and automatic masking. However, the system primarily focuses on enhancing post-production workflows rather than intelligently generating user-guided rough cuts through natural language interaction and human review.

- **Runway ML:** Runway ML is an AI-powered creative platform that offers intelligent video editing features including background removal, object tracking, video generation, and automated visual effects. While the platform significantly simplifies many editing tasks, it emphasizes complete AI automation and cloud-based processing. Users have limited control over the internal editing decisions, and generated results often require additional manual refinement to achieve the desired creative outcome.

4.3 Limitations of Existing System

Although existing video editing applications provide powerful editing capabilities and AI-assisted features, they still exhibit several limitations that reduce editing efficiency and limit user control. The major drawbacks of existing systems are summarized below:

- **Time-Consuming Manual Editing:** Professional video editing software requires editors to manually review large volumes of raw footage, remove unwanted clips, trim silent portions, detect unstable shots, and arrange clips on the timeline. These repetitive tasks significantly increase the time required to produce even a basic rough cut, particularly for long-form videos.
- **Limited Human Control in AI-Based Editing:** Most AI-powered editing applications generate editing suggestions or complete videos automatically with limited opportunities for user intervention. Editors often cannot review or modify individual AI decisions before the final output is produced, reducing creative flexibility and user confidence.
- **Lack of Natural Language Editing Support:** Existing editing software typically requires users to manually configure editing parameters or perform operations through complex interfaces. Very few systems allow editors to describe their desired editing style, clip arrangement, or

target duration using natural language prompts, making the editing process less intuitive for beginners.

- **Isolated AI Features:** Current AI-assisted editing tools generally provide independent features such as scene detection, silence removal, object tracking, or highlight generation. However, these features are not integrated into a unified workflow that intelligently combines video analysis, audio analysis, and user intent to generate a complete AI-assisted rough cut.
- **Limited Transparency and Explainability:** Most AI editing systems function as black-box solutions, providing little explanation for why certain clips are selected, removed, or rearranged. This lack of transparency often forces editors to manually verify and correct AI-generated edits, reducing the overall efficiency gained through automation.

4.4 *Proposed System*

The proposed system, **SmartEdit**, is an AI-assisted Human-in-the-Loop (HITL) video editing application designed to accelerate the traditional video editing workflow while preserving the creative control of the editor. Unlike conventional editing software that requires extensive manual effort or AI-powered tools that perform fully automated editing, SmartEdit combines artificial intelligence with human decision-making to generate an intelligent rough cut that can be reviewed, refined, and finalized by the user. The system integrates natural language understanding, computer vision, audio signal processing, and multimedia processing into a unified desktop application developed using Qt (C++), Python, OpenCV, Librosa, and FFmpeg.

The system allows users to describe their desired editing style and target video duration using natural language prompts. These prompts are interpreted by a Small Language Model (SLM), which converts the user's instructions into structured editing operations. Video analysis is performed

using OpenCV to detect shaky footage, blurred or low-quality clips, while Librosa analyzes the audio track to identify silent portions for automatic trimming. Based on the combined video and audio analysis, the system generates an AI-assisted rough edit that is presented through an interactive timeline interface. Users can preview the generated timeline, accept or reject AI recommendations, manually rearrange clips, and perform additional refinements before exporting the final video. The proposed architecture consists of the following core modules:

- **Prompt Interpretation Module:** This module accepts natural language editing instructions from the user, such as the desired video style, clip arrangement, and target duration. The Small Language Model (SLM) interprets these prompts and converts them into structured editing commands that guide the subsequent AI analysis process.
- **Video Analysis Module:** This module utilizes OpenCV to analyze video frames for detecting shaky footage, blurred scenes, duplicate clips, and other low-quality segments. The extracted visual information is used to identify suitable clips for generating an intelligent rough cut.
- **Audio Analysis Module:** Using the Librosa library, this module analyzes the audio track to detect silent regions and low-energy segments. The detected silent portions are automatically trimmed, reducing repetitive manual editing and improving the overall pacing of the generated video.
- **AI-Assisted Rough Cut Generation Module:** This component combines the results of prompt interpretation, video analysis, and audio analysis to automatically organize clips into a preliminary editing timeline. The generated rough cut follows the editing style and duration specified by the user while eliminating unnecessary footage.
- **Human-in-the-Loop Review Module:** After generating the rough edit, the system presents the timeline to the user for review. Editors can

preview AI-generated edits, accept or reject individual suggestions, rearrange clips, perform manual adjustments, and refine the timeline according to their creative preferences before finalizing the project.

- **Video Rendering and Export Module:** This module utilizes FFmpeg to process the finalized timeline, encode the edited video, and export it in the desired output format while maintaining high video quality and efficient rendering performance.

4.5 Advantages of Proposed System

The proposed **SmartEdit** system offers several advantages over conventional video editing applications by combining Artificial Intelligence (AI) with a Human-in-the-Loop (HITL) workflow. The system is designed to reduce repetitive manual editing tasks while ensuring that the editor retains complete creative control throughout the editing process. The major advantages of the proposed system are as follows:

- **Reduced Video Editing Time:** SmartEdit automates repetitive tasks such as detecting shaky footage, removing silent portions, trimming unnecessary clips, and generating an initial rough cut. This significantly reduces the time required to produce the first version of an edited video.
- **Human-in-the-Loop Editing Workflow:** Unlike fully automated AI editors, SmartEdit allows users to preview, accept, reject, or modify AI-generated editing suggestions. This ensures that the final creative decisions always remain under the control of the editor.
- **Natural Language-Based Editing:** The system enables users to specify the desired editing style, clip arrangement, and target video duration using simple natural language prompts. This makes the editing process more intuitive and accessible, especially for users with limited technical editing experience.

- **Integrated Video and Audio Analysis:** By combining OpenCV for computer vision and Librosa for audio signal processing, the system simultaneously analyzes visual and audio content to identify low-quality footage and silent segments, resulting in a more efficient and intelligent editing workflow.
- **Interactive Timeline for Manual Refinement:** The AI-generated rough cut is presented through an editable timeline interface where users can rearrange clips, perform additional trimming, and refine the sequence before exporting the final video.
- **Scalable and Modular Architecture:** The hybrid architecture developed using Qt (C++), Python, OpenCV, Librosa, and FFmpeg enables easy integration of future AI features such as scene detection, object recognition, subtitle generation, facial recognition, and advanced editing assistance without significant changes to the core system.

4.6 Functional Requirements

The proposed SmartEdit system is designed to accelerate the traditional video editing workflow by combining Artificial Intelligence (AI) with a Human-in-the-Loop (HITL) editing approach. The system provides intelligent video and audio analysis, natural language-based editing, automated rough-cut generation, and manual refinement through an interactive timeline. The functional requirements describe the expected behavior and capabilities of the system.

4.6.1 Video Import and Project Creation

The system shall allow users to create a new editing project and import one or more video files from local storage. The uploaded videos shall be validated to ensure they are supported multimedia formats before processing. The system shall display descriptive error messages for unsupported file formats or corrupted video files. After successful import, video metadata such as

duration, resolution, frame rate, and audio information shall be extracted for further processing.

4.6.2 Prompt-Based Editing

The system shall provide a natural language interface through which users can describe their editing requirements. Users shall be able to specify:

- Desired video style (e.g., cinematic, vlog, interview, travel).
- Target duration of the final video.
- Preferred clip arrangement.
- Editing preferences such as removing silent portions or shaky footage.

The system shall interpret the prompt using a Small Language Model (SLM) and convert it into structured editing instructions for the AI modules.

4.6.3 Video Analysis

The system shall analyze uploaded videos using OpenCV to extract visual information required for intelligent editing. The analysis shall include:

- Detection of shaky footage.
- Detection of blurred or low-quality frames.
- Identification of duplicate or unnecessary clips.
- Extraction of scene and frame information required for rough-cut generation.

The system shall process each uploaded video independently before combining the analysis results.

4.6.4 Audio Analysis

The system shall analyze the audio track of each video using Librosa. The analysis shall identify silent regions, low-energy audio segments, and speech activity. Based on the extracted audio features, the system shall automatically recommend silence-based trimming to improve the pacing of the edited video.

4.6.5 AI-Assisted Rough Cut Generation

After completing prompt interpretation, video analysis, and audio analysis, the system shall automatically generate a rough-cut timeline. The generated timeline shall:

- Arrange clips according to the user's editing prompt.
- Remove unnecessary footage.
- Exclude detected silent portions.
- Prioritize stable and visually suitable clips.
- Produce a rough edit matching the requested duration.

The generated timeline shall be available for user review before final editing.

4.6.6 Human-in-the-Loop Review

The system shall present the AI-generated rough edit through an interactive timeline interface. Users shall be able to:

- Preview the generated rough cut.
- Accept AI-generated editing suggestions.
- Reject individual editing decisions.
- Regenerate the rough cut using a different prompt.

The system shall ensure that no AI-generated edit is permanently applied without user approval.

4.6.7 Manual Timeline Editing

Following user approval of the rough cut, the system shall provide timeline editing functionality. Users shall be able to trim clips, rearrange their sequence, remove or insert clips, and perform additional manual adjustments before exporting the final project.

4.6.8 Video Rendering and Export

The system shall generate the final edited video using FFmpeg. Users shall be able to export the completed project in supported video formats while preserving video quality and synchronization between video and audio streams. The system shall display export progress and notify users when rendering has been completed successfully.

4.7 Non-Functional Requirements

The non-functional requirements define the quality attributes that ensure the SmartEdit system is efficient, reliable, user-friendly, and scalable. These requirements focus on the overall performance of the application rather than its individual functionalities. Since SmartEdit combines desktop application development with Artificial Intelligence (AI), computer vision, audio processing, and multimedia rendering, the system must provide a responsive editing experience while maintaining accuracy and stability.

4.7.1 Performance

The system shall efficiently process video and audio data to generate AI-assisted rough cuts within a reasonable time. Video playback, timeline interaction, and editing operations shall remain responsive even when processing high-resolution videos. AI modules for prompt interpretation,

video analysis, and silence detection shall execute with minimal latency to support an interactive editing workflow.

4.7.2 Usability

The SmartEdit interface shall provide a clean, intuitive, and user-friendly editing environment suitable for both beginners and professional editors. Users shall be able to create projects, import videos, provide editing prompts, review AI-generated suggestions, and manually refine edits through an easy-to-use timeline interface without requiring extensive technical expertise.

4.7.3 Accuracy and Reliability

The system shall accurately detect shaky footage, silent audio segments, and unnecessary clips using computer vision and audio analysis techniques. AI-generated editing suggestions shall be consistent with the user's prompt while ensuring that no editing decision is permanently applied without user approval. The application shall provide stable and reliable operation throughout the editing process.

4.7.4 Security and Data Privacy

The system shall process user videos securely and ensure that project files remain accessible only to authorized users. Video files, editing projects, and AI-generated data shall not be modified or deleted without user confirmation. If cloud-based AI services are integrated in future versions, secure communication protocols shall be used to protect user data during transmission.

4.7.5 Maintainability and Extensibility

The SmartEdit system shall follow a modular software architecture that separates the desktop application, AI processing modules, multimedia processing engine, and user interface. This modular design will simplify maintenance and enable future enhancements such as automatic subtitle

generation, object detection, facial recognition, scene understanding, and advanced AI-assisted editing features without major architectural modifications.

4.7.6 Scalability

The application shall support editing projects containing multiple video files of varying resolutions and durations. The architecture shall be scalable to accommodate additional AI models and multimedia processing capabilities while maintaining acceptable system performance.

4.7.7 Human-in-the-Loop Editing

The system shall ensure that all AI-generated editing suggestions remain under complete user control. Editors shall always have the ability to preview, accept, reject, or manually modify AI-generated rough cuts before exporting the final video. This requirement guarantees that creative decisions remain with the human editor rather than the AI system.

4.8 Feasibility Study

A feasibility study evaluates whether the proposed SmartEdit system can be successfully developed, implemented, and deployed within the available technical resources, operational environment, and project budget. Since SmartEdit integrates Artificial Intelligence, computer vision, audio signal processing, and desktop application development, it is essential to assess its feasibility before implementation. The study is categorized into technical, operational, and economical feasibility.

4.8.1 Technical Feasibility

The SmartEdit system is technically feasible because it is developed using mature and well-supported technologies such as Qt (C++), Python, OpenCV, Librosa, FFmpeg, and Small Language Models (SLMs). These technologies provide reliable support for desktop application development, multimedia

processing, computer vision, and natural language understanding. The modular architecture enables efficient communication between the Qt-based user interface and Python AI modules, while standard desktop hardware is sufficient to execute the proposed editing workflow without requiring specialized computing infrastructure.

4.8.2 Operational Feasibility

Operational feasibility evaluates whether the proposed system effectively satisfies user requirements in real-world editing environments. SmartEdit is designed for content creators, YouTubers, wedding videographers, news editors, and multimedia professionals who frequently perform repetitive editing tasks. By automating rough-cut generation while preserving complete human control through the Human-in-the-Loop workflow, the system integrates naturally into existing editing practices without requiring significant changes to user workflows.

4.8.3 Economical Feasibility

The SmartEdit system is economically feasible because it primarily relies on open-source technologies, including Qt, Python, OpenCV, Librosa, FFmpeg, and Git, thereby reducing software development costs. The hardware requirements are moderate and compatible with standard desktop computers used for video editing. By reducing manual editing time and improving overall productivity, the proposed system offers long-term cost benefits for individual creators, educational institutions, and professional media organizations.

BIBLIOGRAPHY

- [1] T. Tilekbay, A. Kim, D. E. Park, and J. A. Landay, "ExpressEdit: A Multimodal System for Video Editing with Natural Language and Sketching," in *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems (CHI)*, 2024.
- [2] Z. Chen, K. Liang, H. Wang, *et al.*, "EditBoard: A Comprehensive Evaluation Benchmark for Text-based Video Editing," *arXiv preprint arXiv:2402.09257*, 2025.
- [3] K. Wang, Z. Wang, Z. Yuan, *et al.*, "LAVE: An LLM-powered Agent for Video Editing," *arXiv preprint arXiv:2402.19131*, 2024.
- [4] K. Shekar, K. Kasi Viswanadh, V. Srujan Varma, K. Veda Sri, S. Aditya, and S. Sahoo, "AI-Based Intelligent Video Editing System," in *2025 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC)*, 2025.
- [5] S. Tu, Q. Dai, Z. Zhang, S. Xie, Z.-Q. Cheng, C. Luo, X. Han, Z. Wu, and Y.-G. Jiang, "Paper Title," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [6] B. Li, Y. Wu, Y. Lu, J. Yu, L. Tang, J. Cao, W. Zhu, Y. Sun, J. Wu, and W. Zhu, "VEU-Bench: Towards Comprehensive Understanding of Video Editing," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [7] V. Biyyala, B. C. Kathuria, J. Li, and Y. Zhang, "SST-EM: Advanced Metrics for Evaluating Semantic, Spatial and Temporal Aspects in Video Editing," *arXiv preprint*, 2025.
- [8] E. Jones and J. Black, "Integrating AI and User Feedback in Video Editing Software," *IEEE Access*, vol. 12, pp. 10234–10250, 2024.